量化

Quantization

中文修订版·图表与公式随文排版

原作者：@techNmak·Understanding AI 系列·https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Quantization	量化	
Inference	推理	
Precision	精度	
Quantized	量化后的	
Codebook	码本	
Tensor	张量	
Zero-point	零点	
Asymmetric Quantization	非对称量化	
Per-channel	逐通道	
Per-tensor	逐张量	
Per-group	逐组	
Calibration	校准	
Clipping	截断	
Rounding	舍入	
Activation	激活	
Activation Quantization	激活量化	
Model	模型	
Latency	延迟	
Mixed Precision	混合精度	
Group Size	组大小	
Packing	打包	
Transformer Block	Transformer 块	
Transformer	Transformer 模型	Transformer
KV Cache	键值缓存	KV Cache
Representation	表示	
Mental Model	理解框架	
Benchmark	基准测试	

英文原词	中文译名	缩写／符号
Training	训练	
Kernel	内核	
Values	值	
Gradient	梯度	
Token	词元	
Index	索引	
Dynamic Range	动态范围	
Floating-point	浮点	
Floating Point	浮点	
Post-training Quantization	训练后量化	PTQ
Scaling	缩放	
Dequantization	反量化	
Symmetric Quantization	对称量化	
Decode	解码	
Outlier	离群值	
Quantization Error	量化误差	
Low-bit	低比特	
Keys	键	
Accuracy	准确性	
Models	模型	
Matrix	矩阵	
Weight-only Quantization	仅权重量化	
Matrix Multiplication	矩阵乘法	
Head Dimension	头维度	
Attention	注意力机制	Attention
KV Heads	键值头	
Accumulator	累加器	
Core	核心	
Synchronization	同步	

英文原词	中文译名	缩写／符号
Performance	性能	
Quantization-aware Training	量化感知训练	QAT
Forward Pass	前向传播	
Fine-tuning	微调	
Pipeline	流水线	
Matmul	矩阵乘法	
Memory Traffic	内存流量	
Tensor Shape	张量形状	
Combine	合并	
Memory Footprint	内存占用	
Bandwidth-bound	带宽受限	
Throughput	吞吐量	
Accumulation	累加	
Normalization	归一化	
Embeddings	嵌入向量	
FP32	32 位浮点	FP32
Dtype	数据类型	
Vector	向量	
Global Memory	全局内存	
Operators	算子	
Vocabulary	词表	
Layer Normalization	层归一化	LayerNorm
Softmax	Softmax 归一化	Softmax
Operator	算子	
Device Memory	设备内存	
Cache Payload	缓存有效载荷	
Tensor Payload	张量有效载荷	
Tokens	词元	
Materialization	物化	

英文原词	中文译名	缩写／符号
Serving	推理服务	
Allocator Overhead	分配器开销	
Effective Bits	有效位数	
GEMM	通用矩阵乘法	GEMM
Tensor Cores	张量核心	
Launch Overhead	启动开销	
Batch Size	批大小	
Resident	驻留	
Heads	头	
Generation	生成	

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

量化 · 第 8 页

- 阅读导引 · 第 9 页

1 - 什么是量化（Quantization） · 第 10 页

2 - 低精度（Precision）不是一回事 · 第 11 页

3 - 仿射整数量化（Quantization） · 第 12 页

4 - 对称与不对称 · 第 13 页

5 - 工作示例 I：对称 INT8 · 第 14 页

6 - 工作示例 II：不对称 UINT8 · 第 15 页

7 - 舍入（Rounding）、裁剪和误差 · 第 16 页

8 - 量化（Quantization）粒度 · 第 17 页

9 - 工作示例 III：粒度 · 第 18 页

10 - 什么可以量化？ · 第 19 页

11 - 校准（Calibration）和观察者 · 第 20 页

12 - 静态 VS 动态 PTQ · 第 21 页

13 - 量化感知训练（Quantization-aware Training） · 第 22 页

14 - 仅整数与去量化执行 · 第 23 页

15 - 仅权重量化（Weight-only Quantization） · 第 24 页

16 - 为什么“4 位模型（Model）”不是 4 位计算 · 第 25 页

17 - 激活（Activation）异常值 · 第 26 页

18 - LLM.INT8() · 第 27 页

19 - 平滑量化 · 第 28 页

20 - 零定量 · 第 29 页

21 - GPTQ · 第 30 页

22 - 预警 · 第 31 页

23 — GPTQ、AWQ、SmoothQuant 与 LLM.INT8 · 第 32 页

正态分布权重的 4 位码本（Codebook） · 第 34 页

25 - QLORA 和双量化（Quantization） · 第 35 页

26 - FP8：E4M3 和 E5M2 · 第 36 页

27 - FP8 缩放（Scaling）和混合精度（Mixed Precision） · 第 37 页

28 - 块级 FP8 和 FP4 · 第 38 页

29 - KV 缓存量化（Quantization） · 第 39 页

30 - 基维 · 第 40 页

31 - 科维量子 · 第 41 页

32 - 内存核算 · 第 42 页

33 - 工作记忆示例：组元数据 · 第 43 页

34 - 内核和硬件 · 第 44 页

35 - 延迟（Latency）、吞吐量（Throughput）和带宽 · 第 45 页

36 - 透明的 Pytorch 量化器 · 第 46 页

37 - 常见的实施错误 · 第 47 页

38 - 完整的理解框架（Mental Model） · 第 48 页

参考文献 · 第 49 页

量化

对应原稿第 1 页

理解

量化（Quantization）

从刻度和零点到 INT4、FP8、

和现代 LLM 推理（Inference）

面向人工智能工程师、开发人员和认真学习者的简短技术手册

@techNmak

手册10-理解AI系列

- 阅读导引

对应原稿第 2 页

本手册将澄清哪些问题

量化（Quantization）通常被概括为“使用更少的比特”。这在方向上是正确的，但在技术上是不完整的。一个有用的解释必须说明什么张量（Tensor）被量化，使用什么数字密码本（Codebook），如何选择尺度参数，什么被舍入或剪裁，哪些操作实际上以低精度（Precision）运行，以及哪些成本保持在较高的精度。

到最后，您应该能够解释：

• 仿射量化/反量化方程以及标度和零点（Zero-point）的作用；

• 对称量化与非对称量化（Asymmetric Quantization）以及每张量与每通道/每组粒度；

• 为什么舍入（Rounding）、裁剪、校准（Calibration）和异常值会产生不同的误差模式？

• 仅权重、权重激活（Activation）和 KV 缓存量化（Quantization）作为不同的部署问题；

• PTQ、QAT、伪量化（Quantization）以及动态与静态激活量化（Activation Quantization）；

• 为什么“4 位模型（Model）”并不意味着每个值或每个算术运算都是4 位的？

• LLM.int8()、SmoothQuant、GPTQ、AWQ、NF4/QLoRA、FP8、KIVI 和 KVQuant 如何攻击不同的瓶颈；

• 为什么较低的位宽可以减少内存，但不会自动减少实际运行延迟（Latency）；

• 硬件内核、打包（Packing）、组大小（Group Size）、比例和混合精度（Mixed Precision）如何决定实际系统结果。

先备知识

熟悉向量、矩阵、Transformer 块（Transformer Block）以及权重、激活和 键值缓存（KV Cache）之间的差异就足够了。早期关于 键值缓存和 Transformer 块的手册是有用的上下文。

适用范围

本手册是关于数字表示（Representation）和执行的。它不会成为 CUDA 内核（Kernel）手册、完整的混合精度（Precision）训练（Training）指南或基准目录。仅当硬件特定行为改变理解框架（Mental Model）时才包含在内。

研究依据：[1]-[17]

1 - 什么是量化（Quantization）

对应原稿第 3 页

用较小的可表示集替换较大的数值集

模型（Model）通常以某种有限的数字格式存储或计算值。量化（Quantization）将这些值映射到较小的一组可表示级别。一个简单的标量量化器可以写成

$$q = Q(x), \qquad \hat{x} = D(q),$$

其中 q是存储的低精度（Precision）代码，$\hat{x}$是为以后计算而重构的值。

对于神经网络，x可能是权重、激活（Activation）、梯度（Gradient）、优化器状态或 KV 缓存元素。这些对象具有不同的分布和对误差的敏感度不同，因此“模型（Model）被量化”并不是一个完整的规范。

因此，一个有用的部署描述至少需要四个部分：

1. 目标：权重、激活、键值缓存（KV Cache）或其他张量（Tensor）；

2. 格式：INT8、INT4、FP8、NF4、FP4或其他码本（Codebook）；

3. 粒度：张量（Tensor）、通道、词元（Token）、组或块的一个尺度；

4.执行：实际内核（Kernel）如何使用打包值。

在普通情况下，量化（Quantization）是有损的，因为多个高精度（Precision）值映射到相同的代码。工程目标不是准确保留每个数字。它是在减少内存、带宽或计算成本的同时保留重要的模型（Model）行为。

理解框架（Mental Model）。位宽只是量化（Quantization）方案的一维。映射、尺度粒度、校准（Calibration）规则和核同样重要。

研究依据：[1]、[2]

2 - 低精度（Precision）不是一回事

对应原稿第 4 页

整数量化（Quantization）、浮点（Floating-point）格式和码本不同

短语“8 位”可以在数字上描述非常不同的表示形式。

示例	代表理念	值的间距
INT8 仿射	整数代码加刻度/零点	在固定尺度下统一

FP8

符号、指数、尾数不一致；动态范围（Dynamic Range）来自指数

4-bit index into a fixed
non-uniform codebook

固定非均匀码本（Codebook）的 4 位索引（Index）

NF4

非均匀分位数式水平

BF16/FP16

减少浮点（Floating-point）浮点间距，通常被视为混合精度（Mixed Precision）而不是

比 PTQ

这种区别很重要，因为两种“8 位”方法可能具有不同的动态范围、误差

分发、元数据、硬件支持和算术路径。

BF16 和 FP16 通常被讨论为降低精度（Precision）的浮点（Floating-point）格式，而不是训练后量化（Post-training Quantization）方案。 FP8 更接近边界：现代训练（Training）/推理（Inference）系统使用围绕 FP8 张量的缩放（Scaling）配方，因此通常在与量化（Quantization）相同的低精度工具箱中讨论它。

同样，NF4 也不仅仅是“不同规模的 INT4”。它是一个 4 位码本（Codebook），围绕 QLoRA 中的标准化正常权重假设而设计。当前的 bitsandbytes 代码明确将 NF4 视为查找码本，而不是符号/指数/尾数浮点格式。

重要。切勿仅从位宽推断格式的数字行为。“4 位”或“8 位”不足以重构量化器。

研究依据：[8]、[9]、[14]、[16]

3 - 仿射整数量化（Quantization）

对应原稿第 5 页

比例加零点（Zero-point）将实数值映射为整数代码

常见的均匀仿射量化器表示的实值大约为

$$x \approx s(q - z),$$

其中 $s > 0$是刻度， z是整数零点（Zero-point）， q是$[q_{\min}, q_{\max}]$.中的整数代码

然后量化（Quantization）和反量化（Dequantization）

$$q = \operatorname{clamp}\left(\operatorname{round}\left(\frac{x}{s}\right) + z,\ q_{\min}, q_{\max}\right),$$

圆形

$$\hat{x} = s(q - z).$$

比例决定了相邻可表示实值之间的距离。零点（Zero-point）确定哪个整数代码代表真正的零。舍入（Rounding）产生有限精度（Precision）；钳位处理超出所选可表示范围的值。

对于非对称最小/最大映射，一个共同的起点是

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z \approx q_{\min} - \frac{r_{\min}}{s},$$

with z四舍五入并剪裁到整数范围。

确切的约定因运行时而异。一些对称 INT8 实现有意使用$[-127, 127]$而不是完整签名的$[-128, 127]$范围，以保持零附近的精确对称性。

理解框架（Mental Model）。仿射量化（Quantization）是实值区间和有限整数网格之间的坐标变换。

研究依据：[1]、[2]、[13]

4 - 对称与不对称

对应原稿第 6 页

我们需要一个习得的零点（**Zero-point**）吗？

在对称量化（Symmetric Quantization）中，实数范围以零为中心。对于有符号整数，零点（Zero-point）通常固定为零，并且比例由最大绝对值决定：

$$s = \frac{\max |x|}{q_{\max}}, \qquad z = 0.$$

这很简单，并且对于分布近似居中的权重来说通常很有吸引力。

在非对称量化（Asymmetric Quantization）中，观察到的最小值和最大值可以映射到整数范围的不同端。零点（Zero-point）移动网格，以便仍然可以表示真正的零。这个可以

当张量强烈偏离零时，可以更有效地使用可用代码范围。

	对称	不对称
Zero-point	typically fixed at 0	derived from range
Best intuition	centered range	shifted range
Metadata	simpler	includes nonzero z
Common use	many weight schemes	偏移量很重要的激活/KV 方案

没有普遍规则规定权重必须对称或激活不对称。选择

零点（Zero-point） 最佳直觉 元数据 常用

通常固定在 0 中心范围 更简单的许多权重方案

从范围移位范围导出

包括非零 z

取决于分布、后端和量化（Quantization）方法。

重要。非零零点（Zero-point）不是“偏差”。它是用于将整数代码解码（Decode）回近似实数值的数字坐标系的一部分。

研究依据：[1]、[2]、[13]

5 - 工作示例 I：对称 INT8

对应原稿第 7 页

量化、存储然后重建

采取

$$x = [-1.2, -0.5, 0, 0.3, 1.0]$$

并使用带符号的对称 INT8 和代码$[-127, 127]$.

最大星等是1.2,所以

$$s = \frac{1.2}{127} \approx 0.00944882.$$

当 z= 0时，

$$q = \text{round}(x/s) = [-127, -53, 0, 32, 106].$$

反量化（Dequantization）给出

$$\hat{x} = sq \approx [-1.2000, -0.5008, 0, 0.3024, 1.0016].$$

因此，重建误差 (x_hat - x) 近似为

$$[0, -0.0008, 0, +0.0024, +0.0016].$$

在这个小例子中可以看到三件事。首先，当从最大幅度导出尺度时，可以几乎准确地表示端点。其次，内部值移动到最近的网格点。第三，通过舍入到最接近的值，未裁剪的标量最多被替换大约半个量化（Quantization）步长（除了打破平局的惯例）。

如果异常值迫使尺度变得更大，则对于共享该尺度的每个较小值，网格将变得更粗糙。这就是粒度和异常值处理在 LLM 量化（Quantization）中如此重要的原因之一。

理解框架（Mental Model）。量化误差（Quantization Error）并不神秘：它是原始值和最近允许的重建级别之间的距离，加上任何剪切误差。

研究依据：[1]、[2]

6 - 工作示例 II：不对称 UINT8

对应原稿第 8 页

移位范围使用非零零点（Zero-point）

采取

$$x = [-0.4, 0.24, 0.96, 1.6]$$

并将观察到的范围映射到 UINT8 代码$[0, 255]$.

范围宽度为2.0,所以

$$s = \frac{1.6 - (-0.4)}{255} = \frac{2}{255} \approx 0.00784314.$$

2

255

零点（Zero-point）是

$$z \approx 0 - \frac{-0.4}{s} = 51.$$

量化（Quantization）给出

$$q = \text{round}(x/s + 51) = [0, 82, 173, 255].$$

重建给出

$$\hat{x} = s(q - 51) \approx [-0.4000, 0.2431, 0.9569, 1.6000].$$

非零零点（Zero-point）让整数代码 51 代表真正的零，即使观察到的范围不是围绕零对称的。这是非对称仿射量化（Quantization）存在的核心原因。

运行时可以使用校准（Calibration）统计数据而不是精确的最小值和最大值来选择范围。剪切少量极值有时可以减少总误差，因为它可以在大多数值所在的区域上形成更精细的网格。

重要。“使用最小值和最大值”是一个简单的校准（Calibration）规则，而不是法律。实用的量化器可以优化限幅阈值或使用直方图、百分位数、学习范围或特定于方法的目标。

研究依据：[1]、[12]、[13]

7 - 舍入（Rounding）、裁剪和误差

对应原稿第 9 页

三种不同的失真源

量化误差（Quantization Error）通常被作为一件事来讨论，但至少应该将三种机制分开。

舍入（Rounding）误差。可表示区间内的值被移动到最近的网格/码本（Codebook）级别。

剪辑错误。超出选定范围的值将饱和到端点。这可能比一个量化（Quantization）步长大得多。

尺度共享误差。一组异质的价值观共享一个尺度。大的异常值可能会迫使许多其他较小的值采用粗步长。

这就产生了一个核心的权衡。更宽的可表示范围减少了削波，但增加了级别之间的间距。较窄的范围可提高常见值的分辨率，但会限制更极端的值。

对于均匀量化（Quantization），近似步长大小与

范围 宽度 代码数量.

$$\Delta \propto$$

位宽减半可显着减少级别数量：8 位可提供 256 个代码，而 4 位仅可提供 16 个代码。因此，在低位宽时，尺度选择和分布形状更为重要。

GPTQ、AWQ、SmoothQuant 和 OmniQuant 等方法的部分不同之处在于它们如何减少这些错误的下游影响，而不是仅仅在各处应用朴素的舍入到最近值。

理解框架（Mental Model）。低位量化（Quantization）是一个错误分配问题：决定哪些精度（Precision）很重要，哪些可以容忍近似。

研究依据：[5]、[6]、[7]、[12]

8 - 量化（Quantization）粒度

对应原稿第 10 页

一个缩放因子可以表示多少个值?

量化器可以在不同范围内共享参数。

粒度	一个秤涵盖	权衡
每张量	整个张量	元数据最少，适应性最弱

每通道一个通道/行/列更适合，更多尺度 每组固定大小的权重组在低位 LLM 权重中常见妥协 与一个词元（Token）关联的每个词元值

适应词元（Token）变化的激活/缓存

每块硬件定义的块可实现本地扩展和硬件友好

formats

The axis matters. “Per-channel” is incomplete unless we say which dimension defines the channel.

轴很重要。除非我们说哪个维度定义了通道，否则“每通道”是不完整的。

格式

权重矩阵可以根据方法/内核（Kernel）沿输入或输出维度进行分组。

KV-cache方法可以为键和值选择不同的粒度。

更细的粒度通常会减少量化误差（Quantization Error），因为每个尺度覆盖更均匀的子集。但它也会增加规模元数据，并使打包（Packing）、矢量化或内核（Kernel）设计变得复杂。

这就是为什么组大小（Group Size）不仅仅是一个精度旋钮。它也是一个系统参数。

重要。都被描述为“INT4”的两个模型（Models）可能表现不同，因为一个使用 32 个权重的每组尺度，另一个使用 128 个权重，或者因为一个存储零点，而另一个是对称的。

研究依据：[2]、[5]、[6]、[10]、[11]、[14]-[16]

9 - 工作示例 III：粒度

对应原稿第 11 页

每通道规模可以保护小通道免受大通道的影响

考虑一个两行权重矩阵（Matrix）

$$W = \begin{bmatrix} -0.9 & -0.2 & 0.7 \\ -12 & 3 & 8 \end{bmatrix}.$$

3 8

整个张量（Tensor）有一个对称的 INT8 尺度，

$s_{\text{tensor}} = 12/127 \approx 0.09449.$

第一行大约重构为

$$[-0.9449, -0.1890, 0.6614].$$

它相对较小的值被强制放到由第二行中的 12 确定的网格上。

每行一个刻度，

$$s_1 = 0.9/127 \approx 0.00709, \qquad s_2 = 12/127 \approx 0.09449.$$

第一行大约变成

$$[-0.9000, -0.1984, 0.7016].$$

对于这个玩具矩阵（Matrix），均方重建误差从大约

$$8.62 \times 10^{-4} \quad \text{to} \quad 2.59 \times 10^{-4}.$$

确切的数字并不是关于每通道量化（Quantization）的普遍说法。他们展示了一种机制：分离组可以防止一个高动态范围通道设置另一个通道的精度（Precision）。

理解框架（Mental Model）。更细的粒度需要花费更多的元数据来购买本地更好的分辨率。

研究依据：[1]、[2]、[13]

10 - 什么可以量化?

对应原稿第 12 页

权重、激活和 键值缓存（**KV Cache**）具有不同的生命周期

三个重要的 LLM 目标很容易混淆。

权重是跨请求重用的模型（Model）参数。主要是仅权重量化（Weight-only Quantization）

减少模型（Model）存储和权重带宽。

激活是执行模型（Model）时产生的中间张量。它们的分布随着输入、层、词元（Token）和执行阶段的变化而变化。激活量化（Activation Quantization）可以实现低精度（Precision）矩阵乘法（Matrix Multiplication），但它必须处理运行时变化和异常值。

键值缓存（KV Cache）是自回归推理（Inference）期间持久的每个请求注意状态。它的内存随着缓存序列长度、批量/并发、层、KV 头、头维度（Head Dimension）和每个元素的字节数而增长。



模型（Model）生命周期 操作生命周期 请求生命周期

W4A16 模型（Model）、W8A8 模型和 FP8-KV 缓存模型解决不同的资源问题，即使它们都被称为“量化 LLMs”。

重要。始终明确写入量化（Quantization）目标。 W4A16 表示 4 位权重，16 位激活；累加器（Accumulator）精度（Precision）由内核（Kernel）单独定义。它与 W4A8 或 4 位 键值缓存（KV Cache）不是同一系统。

研究依据：[3]-[11]、[15]

11 - 校准（Calibration）和观察者

对应原稿第 13 页

部署前我们如何选择尺度？

训练后量化（Post-training Quantization）通常需要代表性数据来估计范围、裁剪阈值、激活（Activation）统计或重建目标。这个过程通常称为校准（Calibration）。

一个简单的观察者可以跟踪跑步的最小值和最大值。基于直方图的观察者可以使用更详细的标准来估计分布并选择范围。每个通道的观察者收集不同通道的单独统计数据。

> PyTorch 的旧版 torch.ao.quantization API 仍然公开最小/最大、移动平均、每通道和直方图观察器。目前量化（Quantization）开发集中在torchao； LLM 方法使用的精确校准（Calibration）策略可以更加专门化。

校准（Calibration）数据不一定是原始训练（Training）数据。 GPTQ、AWQ、SmoothQuant 和 OmniQuant 都以不同的方式使用小型校准集/统计数据：

• GPTQ 在重建过程中使用由近似二阶信息通知的校准（Calibration）输入；

• AWQ 使用激活（Activation）统计数据来识别/保护显着权重通道；

• SmoothQuant 使用激活（Activation）幅度来计算激活和权重之间的等效重新缩放；

• OmniQuant 优化了校准（Calibration）样品的裁剪和等效转换。

校准（Calibration）不良可能会产生技术上有效的量化模型（Model），其规模与实际工作负载不匹配。

理解框架（Mental Model）。校准（Calibration）不是“测试准确性（Accuracy）”。它是根据代表性数值行为估计量化器参数的过程。

研究依据：[5]-[7]、[12]、[13]

12 - 静态 VS 动态 PTQ

对应原稿第 14 页

何时选择激活量化（**Activation Quantization**）参数?

经典量化（Quantization）术语区分了两种常见的训练（Training）后模式。

静态 PTQ 在部署之前使用校准（Calibration）通道来确定激活量化（Activation Quantization）参数。在推理（Inference）时，运行时会重用这些参数。

动态量化（Quantization）通常提前存储量化权重，但从当前运行时张量（Tensor）导出激活（Activation）尺度。这会适应输入变化，但会增加运行时工作。

LLM 文献有时使用不同的特定于方法的语言，因此这些术语不应机械地应用于每个现代内核（Kernel）。仅权重 GPTQ/AWQ 模型（Models）、FP8 缩放（Scaling）配方和 KV 缓存量化器都有自己的缩放更新规则。

核心系统的权衡仍然有用：

预先计算的统计数据

↔

运行时适配.

运行时适应可以改善对当前张量（Tensor）的拟合，但如果内核不是围绕它们设计的，则缩放计算、缩减操作、转换和同步（Synchronization）可能会消除部分性能（Performance）优势。

重要。“动态”并不意味着权重本身被重新训练或改变。它指的是运行时值的量化（Quantization）参数何时确定。

研究依据：[1]、[13]、[14]

13 - 量化感知训练（Quantization-aware Training）

对应原稿第 15 页

在模拟量化器的同时进行训练

训练后量化（Post-training Quantization）会修改训练后的模型（Model）。相反，量化感知训练（Quantization-aware Training）(QAT) 使优化过程受到量化（Quantization）效应的影响。

典型的 QAT 前向传播（Forward Pass）使用假量化（Quantization）：值被量化为离散代码并立即反量化回浮点（Floating-point），因此网络会经历舍入（Rounding）/裁剪行为，而训练（Training）机器仍然可以更新浮点参数。

从概念上讲，

$x \to q = Q(x) \to \hat{x} = D(q) \to$网络其余部分.

回合操作通常不可微分，因此实用的 QAT 使用代理梯度（Gradient）行为，例如直通估计器或特定于框架的假量化模块。

QAT 可以在困难的位宽下恢复精度，因为模型（Model）围绕量化器进行学习。代价是它需要训练（Training）/微调（Fine-tuning）管道、数据和计算。因此，当质量仍然可以接受时，大型模型部署通常更喜欢 PTQ。

雅各布等人。将量化（Quantization）与训练（Training）相结合，以实现仅整数算术推理（Inference），而当前的 PyTorch API 为 QAT 工作流程公开了假量化和观察者模块。

理解框架（Mental Model）。 PTQ 问“这个经过训练（Training）的模型（Model）在量化器中的生存能力如何？”QAT 问“训练能否使模型适应量化器？”

研究依据：[1]、[13]

14 - 仅整数与去量化执行

对应原稿第 16 页

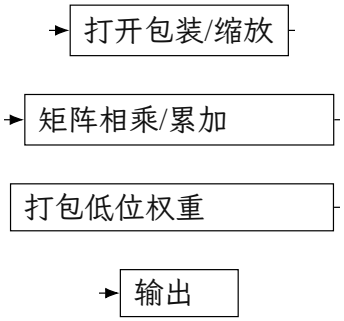
存储格式不唯一决定算术格式

量化张量（Tensor）可以参与多个执行路径。

在整数管道中，输入和权重可以表示为整数，矩阵乘法（Matrix Multiplication）累加为更宽的整数类型，比例因子将结果转换到下一个数值域。雅各布等人。开发了这种仅用于整数算术推理（Inference）的风格。

在许多 LLM 仅权重内核（Kernel）中，从内存中读取打包的低位权重，在内核内部解包/反量化，并与更高精度（Precision）的激活相乘。即使算术路径不是“到处都是 INT4”，内存流量（Memory Traffic）也是低位的。

现代硬件还可以为 INT8、FP8 或 FP4 样式格式提供本机低精度（Precision）矩阵（Matrix）引擎。哪条路径最快取决于设备支持、张量形状（Tensor Shape）、打包（Packing）、内核（Kernel）融合以及是否隐藏转换开销。



重要。检查点的存储位宽、内核（Kernel）输入位宽、乘法器格式和累加器（Accumulator）格式都可以不同。

研究依据：[1]、[4]、[9]、[14]、[15]

15 - 仅权重量化（Weight-only Quantization）

对应原稿第 17 页

压缩模型（Model）而不强制激活较低

仅权重量化（Weight-only Quantization）对于自回归 LLM 推理（Inference）很有吸引力，因为模型（Model）权重

很大并且通过矩阵（Matrix）乘法重复流式传输，尤其是在小解码（Decode）批大小时。

W4A16 等表示（Representation）法通常意味着权重以 4 位表示形式存储，而激活值保持在 FP16/BF16 精度（Precision）左右。内核（Kernel）可以加载打包的 4 位权重，在芯片上对它们进行反量化/缩放，并将它们与更高精度的激活相结合。

好处包括：

• 更低的检查点和设备内存占用（Memory Footprint）；

• 每个词元（Token）的权重带宽较低；

• 安装在固定设备上的较大型号；

• 当内核（Kernel）受带宽限制并经过优化时，潜在的延迟（Latency）/吞吐量（Throughput）增益。

但仅权重量化（Weight-only Quantization）不会缩小激活（Activation）内存、KV 缓存或每个中间张量（Tensor）。它也不能保证成比例的计算加速。

GPTQ 和 AWQ 是两个著名的低位仅权重系列。它们的不同之处在于如何选择量化权重/比例，但都依赖专门的内核将压缩转换为部署性能（Performance）。

理解框架（Mental Model）。仅权量化攻击用于表示和移动模型（Model）参数的字节。它不会量化整个推理（Inference）状态。

研究依据：[6]、[7]、[15]

16 - 为什么“4 位模型（Model）”不是 4 位计算

对应原稿第 18 页

存储、元数据和积累是口号

假设模型（Model）名义上具有 4 位权重。仍然可以保留几个更高精度（Precision）的物体：

• 每组的缩放因子和可能的零分；

• 偏差和标准化参数；

• 嵌入向量（Embeddings）或选定的层保持更高的精度（Precision）；

• FP16/BF16 中的激活；

• INT32、FP16、BF16 或 FP32 中的累加器取决于内核（Kernel）；

• 另一种数据类型（Dtype）的键值缓存（KV Cache）；

• 内核（Kernel）内的临时反量化片段。

QLoRA 是说明这种语言为何如此重要的一个清晰示例。它将冻结的基础模型（Model）存储在 4 位 NF4 表示（Representation）中，但训练（Training）计算使用更高精度（Precision）的计算数据类型（Dtype），并且梯度流到 LoRA 适配器中，而不是将整个训练过程转变为 4 位算术。

同样，W4A16 TensorRT-LLM 配方明确指定了权重和激活的不同精度。

因此，“4 位模型（Model）”一词最好理解为特定存储/量化（Quantization）合约的简写，而不是声称内存中的每个数字和每个乘法都是四位宽。

重要。在比较量化系统时，要求提供完整的精度（Precision）元组：权重、激活、累加器、键值缓存（KV Cache）、比例和任何未量化的异常。

研究依据：[8]、[15]、[16]

17 - 激活（Activation）异常值

对应原稿第 19 页

几个大维度可以为许多普通值设置比例

LLM 激活可以包含持久的高强度特征。如果在张量（Tensor）或向量（Vector）之间共享一个尺度，这些异常值会扩展所需的动态范围（Dynamic Range），并使大多数较小值的量化（Quantization）步长变得更粗糙。

这就是为什么激活量化（Activation Quantization）比权重量化（Quantization）更难。激活（Activation）分布也会随着输入和运行时状态而变化，而权重在训练（Training）后是固定的。

几种有影响力的方法有不同的反应：

• LLM.int8() 隔离离群特征尺寸并以更高的精度（Precision）计算它们，而大多数值使用 INT8；

• SmoothQuant 通过等效变换重新调整激活（Activation）和权重，使激活异常值变得更容易量化；

• AWQ 使用激活（Activation）统计来识别量化误差（Quantization Error）最重要的权重通道，然后重新缩放以保护它们。

这些不是可以互换的算法。他们分享了一个教训：矩阵（Matrix）乘法所看到的数值分布比单独的标称位宽更重要。

理解框架（Mental Model）。异常值的代价是昂贵的，因为共享尺度会将局部极端值变成全局分辨率损失。

研究依据：[3]、[5]、[7]

18 - LLM.INT8()

对应原稿第 20 页

将几乎所有矩阵乘法（**Matrix Multiplication**）保留在 **INT8** 中，隔离异常值

LLM.int8() 专为大型 Transformer 推理（Inference）而设计，其中朴素的 INT8 量化（Quantization）会随着模型（Model）规模的增加而降低质量。

该方法结合了两种想法。首先，大多数值使用矢量 INT8 量化（Quantization）以及内积的单独归一化（Normalization）常数。其次，被识别为异常值的特征维度被分离并以更高的精度（Precision）进行处理。该论文报告称，超过 99.9% 的值仍然可以参与 8 位路径，而离群维度则使用 16 位矩阵乘法（Matrix Multiplication）。

这是混合精度（Precision）分解，而不是声称整个模型（Model）是纯 INT8 算术。

系统结果对于简单化的速度主张也是一个有用的警告。该论文的补充基准表明，量化（Quantization）/反量化（Dequantization）开销可以主导较小的矩阵（Matrix）乘法，而较大的模型（Model）维度从 INT8 执行中受益更多。

重要的。 LLM.int8() 的教训比一种算法更广泛：低位算术仅在以硬件感知方式处理数值异常和转换开销时才有帮助。

研究依据：[3]

19 - 平滑量化

对应原稿第 21 页

将量化（**Quantization**）难度从激活转移到权重

SmoothQuant 针对 LLM 矩阵（Matrix）乘法的 W8A8 训练后量化（Post-training Quantization）。其核心观察是权重通常比具有强异常值的激活更容易量化。

对于激活（Activation）通道 X_j和相应的权重通道 W_j,SmoothQuant 引入每通道缩放（Scaling）因子 s_j并应用以下形式的代数等效变换

$$XW = (X\ \mathrm{diag}(s)^{-1})\,(\mathrm{diag}(s)W).$$

全精度（Precision）乘积在量化（Quantization）之前保持不变。然而，从数值上来说，当权重幅度增加时，激活（Activation）幅度可以减小，从而将部分量化难度从激活转移到权重。

平滑强度由特定于方法的统计数据/超参数控制。该变换是根据校准（Calibration）数据离线计算的，因此部署不需要重新训练原始模型（Model）。

SmoothQuant 公布的加速与其 INT8 内核、模型（Models）和硬件相关。持久的概念是等效的重新缩放，而不是通用的延迟（Latency）乘数。

理解框架（Mental Model）。有时，最好的量化器会在舍入（Rounding）之前更改表示（Representation），而不是接受原始张量（Tensor）分布。

研究依据：[5]

20 - 零定量

对应原稿第 22 页

量化（**Quantization**）也是一个后端问题

ZeroQuant 很有用，因为它明确了一个经常被忽视的点：量化（Quantization）算法和高效的量化系统是解决方案的不同层。

该方法将细粒度权重/激活量化（Activation Quantization）与分层知识蒸馏选项和旨在消除量化（Quantization）/反量化（Dequantization）开销的优化后端相结合。在论文的实验中，INT8 权重激活（Activation）推理（Inference）和混合 INT4/INT8 方案相对于 FP16 实现了模型和硬件特定的效率改进。

确切的基准数字并不比解决方案的架构重要：

1.选择量化（Quantization）粒度/范围；

2. 如果需要恢复质量；

3. 实现内核，防止转换开销抹去节省的成本。

如果运行时在全局内存（Global Memory）中反复实现反量化权重、启动额外的内核或回退到不受支持的运算符，则数学上良好的低位表示（Representation）仍然可能很慢。

理解框架（Mental Model）。量化（Quantization）有算法的一半和系统的一半。没有执行路径的压缩不是推理（Inference）优化。

研究依据：[4]

21 - GPTQ

对应原稿第 23 页

使用近似二阶信息的一次性权重量化（Quantization）

GPTQ 是一种训练（Training）后权重量化（Quantization）方法，旨在将非常大的生成 Transformer 压缩到低位宽，而无需完全重新训练。

在较高层次上，它逐层量化（Quantization）权重，同时补偿早期量化权重引入的误差。该更新使用源自校准（Calibration）激活的近似二阶信息，遵循最佳大脑量化谱系，但经过重新组织以使该过程在 LLM 规模上实用。

这与简单地独立地对每个权重进行舍入（Rounding）不同。 GPTQ 尝试在代表性输入下最小化量化（Quantization）对层输出的影响。

原始论文演示了非常大的 GPT/OPT 类型模型（Models）的 3 位和 4 位权重量化（Quantization），并包含自定义的低位内核。现代实现添加了详细信息，例如组大小（Group Size）、激活（Activation）顺序、静态组、打包（Packing）布局和后端特定内核。

重要。“GPTQ 4 位”不是一种通用的二进制格式。准确性（Accuracy）和运行时间取决于组大小（Group Size）、对称/非对称选择、内核（Kernel）实现和校准（Calibration）配置。

研究依据：[6]

22 - 预警

对应原稿第 24 页

使用激活来确定哪些权重通道最重要

AWQ 是另一种训练（Training）后、低位仅权重方法，但其机制与 GPTQ 不同。

该论文观察到，量化误差（Quantization Error）在权重通道中并不同样重要。激活（Activation）统计数据用于识别显着通道，等效的每通道缩放（Scaling）变换可减少重要权重的误差，而无需存储受保护权重的混合精度（Precision）集。

AWQ 刻意避免反向传播和层重建。其校准（Calibration）过程从激活（Activation）统计中搜索缩放（Scaling）因子，然后将生成的权重打包以进行硬件友好的推理（Inference）。

这给出了有用的概念对比：

• GPTQ 使用重建/二阶误差补偿程序；

• AWQ 使用激活（Activation）感知缩放（Scaling）来保护显着权重通道；

• 两者通常都部署为具有专用内核的低位仅权重量化（Weight-only Quantization）。

已发布的 TinyChat 加速特定于本文的设备和实现，而不是孤立的 AWQ 位宽属性。

理解框架（Mental Model）。量化（Quantization）方法决定要避免哪些错误；内核（Kernel）决定压缩表示（Representation）是否会带来性能（Performance）优势。

研究依据：[7]

23 — GPTQ、AWQ、SmoothQuant 与 LLM.INT8

对应原稿第 25 页

相似的词汇，不同的问题

方法 主要目标 核心思想 典型部署框架

			ing
LLM.int8()	W/A matmul	isolate outlier dimen-	mixed INT8/16-bit

LLM.int8()

W/A 矩阵相乘

隔离异常值维度

英

混合 INT8/16 位

西翁

W8A8

平滑量

将激活（Activation）难度迁移到权重

INT8 矩阵相乘

利用二阶信息进行GPTQ权重重构

W3/W4A16型

AWQ 权重激活（Activation）感知通道

W4A16型

内尔缩放

该表是有意设计的概念性的。真正的框架支持多种变体和位宽。

三个区别可以防止混淆：

1. 仅权重与权重激活（Activation）：减少模型（Model）带宽与启用低位激活矩阵乘法不同；

2.算法 vs 格式：GPTQ 和 AWQ 是方法，而不是原始数值数据类型；

3. 量化器与运行时：相同的量化检查点在不同内核和硬件上的执行情况可能不同。

重要。不要问“GPTQ 比 INT4 更好吗？”GPTQ是一种产生低位权重的方法； INT4 描述了一个数字/存储系列。这种比较混合了抽象级别。

研究依据：[3]、[5]-[7]

正态分布权重的 4 位码本（Codebook）

对应原稿第 26 页

24 - NF4

QLoRA 引入了 NormalFloat 4 (NF4)，用于量化预训练权重，这些权重在按局部绝对最大值归一化（Normalization）后近似正态分布。

NF4 使用从正态分布的分位数中选择的 16 个可表示的码本（Codebook）值，并将其标准化为有界范围。因此，存储的 4 位值是该码本的索引（Index）。它不是带有符号、指数和尾数字段的传统浮点（Floating-point）编码。

当前的 bitsandbytes 源明确说明了这一点：NF4 和 FP4 使用不同的码本/编码，即使每个存储索引（Index）都消耗 4 位。

QLoRA 将 NF4 描述为信息理论上对于正态分布权重而言最优的
其规定的结构。后来的技术说明认为严格的最优性并不能实现
有限块absmax归一化，因为归一化后分布取决于块
尺寸。将原始主张视为特定于方法的动机，而不是通用属性。 [8,17]

非统一码本（Codebook）级别（示意图）

理解框架（Mental Model）。 NF4 通过选择根据权重分布假设定制的 16 个非均匀重建值集进行压缩。

研究依据：[8]、[16]、[17]

25 - QLORA 和双量化（Quantization）

对应原稿第 27 页

量化冻结的基础模型（Model），训练小型适配器

QLoRA 将低位存储与参数高效的微调（Fine-tuning）相结合。预训练的基础模型（Model）被冻结并存储在 4 位 NF4 中，而梯度则通过其反量化计算反向传播到可训练的 LoRA 适配器中。

另外两个想法很重要：

双量化（Quantization）对量化常数本身进行量化，从而减少比例元数据开销。

分页优化器使用统一内存分页来管理微调（Fine-tuning）期间的瞬时优化器内存峰值。

计算数据类型（Dtype）保持比 4 位存储更高的精度（Precision）。这就是为什么在没有限定的情况下将 QLoRA 描述为“4 位训练（Training）”会产生误导。

QLoRA 的贡献也是一个有用的提醒，即量化（Quantization）可用于使训练（Training）/微调内存变得可行，而不仅仅是推理（Inference）部署。

重要。 NF4 属于冻结基本重量的存储/量化（Quantization）表示（Representation）。 LoRA 适配器参数、优化器状态、梯度、激活和计算数据类型遵循自己的精度（Precision）规则。

研究依据：[8]、[16]

26 - FP8：E4M3 和 E5M2

对应原稿第 28 页

浮点（Floating-point）量化（Quantization）以精度（Precision）换取动态范围（Dynamic Range）

广泛使用的 FP8 提案定义了两种 8 位浮点（Floating-point）格式：

• E4M3：1 个符号位、4 个指数位、3 个尾数位；

• E5M2：1 个符号位、5 个指数位、2 个尾数位。

E5M2具有更宽的动态范围（Dynamic Range）和更低的有效精度（Precision）。 E4M3在更窄的范围内具有更高的精度。

与一种尺度下的仿射 INT8 不同，FP8 值在实数空间中分布不均匀。指数随幅度改变步长。

最初的 FP8 工作演示了跨神经架构（包括大型语言模型（Models））的训练（Training）和推理（Inference），并使用围绕这些格式的扩展策略。 NVIDIA Transformer Engine 在支持的 GPU 上使用 E4M3/E5M2 配方，通常可以保持更高精度（Precision）的敏感操作。

这就是为什么“FP8模型（Model）”不应该被理解为“每个操作都在FP8中执行”。层归一化（Layer Normalization）、softmax、缩减、累积和不支持的操作可以保留 BF16/FP16/FP32，具体取决于配方。

理解框架（Mental Model）。整数量化（Quantization）使用缩放均匀网格； FP8 使用浮点（Floating-point）网格，其间距随指数变化。

研究依据：[9]、[14]

27 - FP8 缩放（Scaling）和混合精度（Mixed Precision）

对应原稿第 29 页

格式只是 **FP8** 配方的一半

FP8 张量（Tensor）的动态范围（Dynamic Range）有限，因此实际系统会维护将更高精度（Precision）张量映射到可用 FP8 范围的比例信息。

Transformer Engine 公开了多种策略，包括延迟缩放（Scaling）、当前每张量缩放和块式变体。典型的比例源自观察到的绝对最大值 (amax) 和最大可表示 FP8 值，其边际/历史记录取决于配方。

混合精度（Mixed Precision）仍然至关重要。当前的 Transformer Engine 文档指出，线性运算可以低精度（Precision）运行，而归一化（Normalization）、softmax 和其他运算默认保持较高精度；注意力机制（Attention）精度还取决于配方/后端配置。

这种分离解释了为什么低精度（Precision）训练（Training）可以保持稳定性：不同操作的精度花费不同，而不是强制整个图采用一种格式。

重要。 E4M3 等数据类型名称并不完全指定 FP8 训练（Training）/推理（Inference）系统。缩放（Scaling）粒度、amax 策略、累积精度（Precision）和算子（Operator）覆盖范围是秘诀的一部分。

研究依据：[9]、[14]

28 - 块级 FP8 和 FP4

对应原稿第 30 页

现代硬件使尺度更接近数据

当前实施说明（Transformer Engine 2.19，2026 年 9 月）：重新检查当前文档。

较新的加速器一代支持更细粒度的块扩展。例如，当前的 NVIDIA Transformer Engine 文档描述了 Blackwell 级硬件上的 MXFP8，使用 32 个值块的本地缩放（Scaling），并将 NVFP4 作为受支持的 4 位低精度（Precision）配方。

原因很直观。整个张量（Tensor）的一个尺度必须适应其全局动态范围（Dynamic Range）。为每个小块提供自己的尺度可以减少每个码本（Codebook）必须覆盖的范围，这可以提高局部精度（Precision）。

这带有额外的元数据和布局规则。块格式还可能需要不同的量化视图以用于行方向和列方向使用，因为在定向块分组下量化张量（Tensor）和量化其转置在数值上并不相同。

这些格式是特定于硬件和库的，因此它们应该具有版本范围。常青的想法是：

更精细的尺度粒度 ⇒更好的局部拟合+更多元数据/布局约束.

重要。 NVFP4、NF4 和通用 INT4 是三种不同的数值方案。共享“4 位”并不意味着它们可以互换。

研究依据：[14]、[15]

29 - KV 缓存量化（Quantization）

对应原稿第 31 页

压缩每个请求的状态，而不是模型（Model）权重

在自回归解码过程中，键值缓存（KV Cache）会因长上下文和高并发而消耗大量设备内存（Device Memory）。量化 K/V 状态可以减少每个缓存词元（Token）的字节数，还可以减少注意力机制（Attention）读取缓存时的内存带宽压力。

对于传统的缓存有效载荷（Cache Payload），

$$M_{KV} = 2LTH_{KV}D_h b,$$

其中 D_h 是每个头的键/值维度。

因此减少每个元素的存储字节 b直接减少了理想化的张量有效载荷（Tensor Payload）。

但 KV 缓存量化（Quantization）在数值上与权重量化不同。键和值是根据每个请求动态生成的。它们的分布可能因层、头、词元（Token）和模型（Model）架构而异。量化误差（Quantization Error）还为每个后续解码（Decode）步骤提供未来的注意力机制（Attention）计算。

因此，现代运行时公开了单独的 KV 缓存量化（Quantization）模式，包括 FP8 和受支持硬件上更新的低精度（Precision）格式。

理解框架（Mental Model）。权重量化（Quantization）会更改共享模型（Model）资产。 KV 缓存量化改变了不断增长的、特定于请求的推理（Inference）状态。

研究依据：[10]、[11]、[15]

30 - 基维

对应原稿第 32 页

键和值不需要使用相同的粒度

KIVI 研究了缓存键和值的分布，发现不对称性对于极低位量化（Quantization）很有用。

该论文报告称，密钥表现出通道方式的异常结构，从而激发了每个通道的密钥量化（Quantization），而每个词元（Tokens）的值可以得到更好的处理。 KIVI 将这些选择结合在免调整的非对称 2 位 KV 缓存方法中，并在实际实现中为最近的词元以更高的精度（Precision）保留了一个小的残留区域。

这是一个重要的概念课程：即使在注意力机制（Attention）方程中成对出现的张量也可以具有不同的数值统计量，因此也可以具有不同的最佳量化（Quantization）轴。

KIVI 报告的内存和吞吐量（Throughput）改进与其模型（Models）、工作负载、内核（Kernel）实现和实验设置相关。本手册的持久结果是分布驱动的粒度选择。

重要。“将 K 和 V 量化为 2 位”并不是 KIVI 的完整描述。每个通道/每个词元（Token）的不对称性是该方法的核心。

研究主播：[10]

31 - 科维量子

对应原稿第 33 页

长上下文缓存压缩不仅仅使用位宽

KVQuant 是用于长上下文推理（Inference）的极低位 KV 缓存量化（Quantization）的另一项工作。它研究量化感知选择，例如每通道密钥处理、非均匀量化、异常值处理和优化内核。

更广泛的一点是，非常低精度（Precision）的 KV 缓存压缩成为系统协同设计问题：

• 选择保持注意力机制（Attention）质量的数字表示（Representation）；

• 决定每个通道/词元（Token）/组存储哪些统计数据/比例；

• 如果需要，处理异常值或最近的词元（Tokens）；

• 实现注意力机制（Attention）内核，无需昂贵的物化（Materialization）即可消耗压缩的K/V。

如果解压缩成为瓶颈或者注意力机制（Attention）内核（Kernel）无法有效地使用该格式，一种方法可以报告理论缓存的显着减少，但仍然无法提高推理服务（Serving）吞吐量（Throughput）。

KVQuant 发布的“1000 万上下文”框架属于其研究设置，不应被解读为任意模型（Models）或硬件的通用上下文保证。

理解框架（Mental Model）。对于键值缓存（KV Cache），压缩比、注意力机制（Attention）质量和解码（Decode）带宽必须一起评估。

研究依据：[11]

32 - 内存核算

对应原稿第 34 页

理想的位宽节省是整个运行时占用空间的下限

对于每个 b位存储的 P参数，理想的原始有效载荷为

$$M_{\text{weights}} = \frac{Pb}{8} \text{ bytes.}$$

对于示例性的 70 亿参数模型：

有效载荷重量	十进制 GB	吉布
16-bit	14.0	13.04
8-bit	7.0	6.52
4-bit	3.5	3.26

真正的量化检查点/运行时内存更大，因为它们可以包含比例、零

16 位 8 位 4 位

点、对齐/填充、打包（Packing）元数据、未量化层、偏差/规范、码本（Codebook）状态和临时缓冲区。

运行时内存还包括激活、键值缓存（KV Cache）、分配器开销（Allocator Overhead）、CUDA 图形/工作区，以及加载/转换期间可能的多个副本。

重要。 4 位权重有效载荷大约是 16 位权重有效载荷的四分之一。这并不意味着 GPU 内存使用总量是四分之一。

研究依据：[2]、[6]-[8]、[15]、[16]

33 - 工作记忆示例：组元数据

对应原稿第 35 页

“4 位”权重可能会花费超过 4 个有效位

假设对称分组 4 位权重，每 128 个权重有一个 FP16 标度。

各组店：

$128 \times 4 = 512$重量位

加号

16 个刻度位.

所以有效成本是

$$4 + \frac{16}{128}$$

128= 4.125位/重量.

对于 70 亿个重量，理想的有效载荷变为

$$7 \times 10^9 \times \frac{4.125}{8} = 3.609375 \times 10^9 \text{ bytes} \approx 3.36 \text{ GiB}.$$

8

如果该方案还为每组存储一个 FP16 零点（Zero-point），则成本将变为

$$4 + \frac{32}{128}$$

128= 4.25位/重量

在对齐或其他元数据之前。

这仍然远低于 16 位存储，但它说明了为什么“参数计数乘以半个字节”只是一阶估计。

理解框架（Mental Model）。组大小（Group Size）控制数字局部性和元数据摊销。较小的群体通常会提高适合度，同时增加规模/零点（Zero-point）开销。

研究依据：[6]-[8]、[16]

34 - 内核和硬件

对应原稿第 36 页

只有当机器能够有效地消耗压缩时，压缩才会变得速度

低精度（Precision）性能（Performance）取决于完整的执行路径。

一个有用的清单是：

• GPU/加速器是否具有对该格式的本机张量（Tensor）核心/指令支持？

• 权重是否打包在内核（Kernel）期望的布局中？

• 尺度是否有效加载并与反量化（Dequantization）融合？

• 当前批次/阶段的操作是否类似于GEMM 或GEMV？

• 对于不支持的形状/运算符，运行时是否会回退到更高精度（Precision）的实现？

• 转换是在芯片上执行的，还是反量化的张量写回全局存储器？

当前的 TensorRT-LLM 公开了不同的配方，例如 FP8、FP4、W4A16 GPTQ/AWQ、W4A8 变体和量化 KV 缓存模式。 Transformer Engine 同样通过硬件限制来区分 FP8、MXFP8 和 NVFP4 配方。

这就是为什么量化检查点在错误的设备或软件堆栈上可能更小但更慢。

重要。位宽是表示（Representation）的一个属性。速度是表示+算子（Operator）形状+内核+硬件+工作负载的属性。

研究依据：[14]、[15]

35 - 延迟（Latency）、吞吐量（Throughput）和带宽

对应原稿第 37 页

较低的精度（**Precision**）对不同的瓶颈有不同的改变

量化（Quantization）可以帮助多种资源，但并不总是同时帮助。

容量。较小的权重/缓存允许适应更大的模型（Models）或更多的并发请求。

带宽。必须加载更少的字节才能进行权重绑定解码（Decode）或 KV 绑定注意力机制（Attention）。

计算吞吐量（Throughput）。原生低精度（Precision）张量核心（Tensor Cores）每秒可以在支持的形状上执行更多操作。

延迟（Latency）。仅当减少的瓶颈超过打包（Packing）/反量化（Dequantization）/启动开销（Launch Overhead）时，端到端延迟才会改善。

吞吐量（Throughput）。即使单个请求延迟（Latency）几乎没有变化，内存节省也可以提高批处理/并发性。

LLM.int8() 提供了一个具体的历史示例：较小的矩阵（Matrix）乘法可能会损失量化（Quantization）开销，而较大的维度则受益更多。 KIVI 类似地展示了缓存压缩如何增加可行的批大小（Batch Size），即使算术图在其他方面相似，这也会改变吞吐量（Throughput）。

理解框架（Mental Model）。询问移动了哪些资源：驻留（Resident）字节、传输字节、算术吞吐量（Throughput）、批量容量或端到端时间。没有衡量标准的“更快”是不完整的。

研究依据：[3]、[10]、[14]、[15]

36 - 透明的 Pytorch 量化器

对应原稿第 38 页

教学对称量化-反量化

以下代码反映了已工作的 INT8 示例。它是故意简单的，而不是

优化的推理内核。

```
import torch

def symmetric_qdq(x, bits=8):
    qmax = 2 ** (bits - 1) - 1
    scale = x.abs().max() / qmax
    scale = torch.clamp(scale, min=torch.finfo(x.dtype).eps)
    q = torch.clamp(
        torch.round(x / scale), -qmax, qmax
    ).to(torch.int8)
    x_hat = q.float() * scale
    return q, scale, x_hat

x = torch.tensor([-1.2, -0.5, 0.0, 0.3, 1.0])
q, scale, x_hat = symmetric_qdq(x)
# q = [-127, -53, 0, 32, 106]
# x_hat = [-1.2000, -0.5008, 0.0000, 0.3024, 1.0016]
```

对于位 < 8，q 在这里仍然使用 torch.int8；此示例未实现子字节打包。生产实现必须另外定义粒度、数据类型/范围约定、

零点、包装布局、比例数据类型（Dtype）、组大小（Group Size）、裁剪/校准（Calibration）策略、算子（Operator）支持和累积精度（Precision）。

重要。该函数演示了数字地图。它不会仅仅因为张量（Tensor）代码是整数就使 PyTorch 与优化的 INT8 内核（Kernel）执行矩阵乘法（Matrix Multiplication）。

研究依据：[1]、[13]

37 - 常见的实施错误

对应原稿第 39 页

大多数低位错误是合约不匹配

1. 比较无格式的位宽。 NF4、FP4、INT4 和 NVFP4 不可互换。

2. 从内存估计中删除比例元数据。分组方案需要尺度，有时还需要零点。

3. 使用错误的轴。每个通道沿错误的维度可能会改变准确性（Accuracy）和内核（Kernel）兼容性。

4. 重复使用不良校准（Calibration）数据。激活（Activation）范围可能取决于工作负载。

5. 假设存储精度（Precision）等于计算精度。仅权重内核通常以更高的精度进行计算/累加。

6. 忽略未量化的异常。规范、嵌入向量（Embeddings）、输出头或选定的层可以保持高精度（Precision）。

7. 将检查点大小作为延迟（Latency）进行基准测试。较小的文件并不能保证更快的生成（Generation）。

8. 忘记包装/布局。数学上正确的量化器可能无法被目标内核（Kernel）使用。

9. 混合不兼容的尺度/检查点。量化（Quantization）元数据是模型（Model）合约的一部分。

10. 普遍提高出版论文速度。硬件、批次、序列长度和软件版本必须匹配。

理解框架（Mental Model）。将量化（Quantization）检查点视为权重+量化元数据+所需的执行合约。

研究依据：[3]-[16]

38 - 完整的理解框架（Mental Model）

对应原稿第 40 页

量化（Quantization）十题

当您看到量化模型（Model）时，询问：

1.什么是量化？权重、激活、键值缓存（KV Cache），还是其中的几个？

2.什么格式？ INT8、INT4、FP8、NF4、FP4，还有其他吗？

3.什么映射？均匀仿射、浮点（Floating Point）还是非均匀码本（Codebook）？

4.什么粒度？张量（Tensor）、通道、组、词元（Token）还是块？

5. 缩放因子如何选择？校准（Calibration）、运行时 amax、重建、学习剪裁？

6. 异常值会怎样？剪辑、隔离、重新缩放、保护或保持高精度（Precision）？

7. 什么实际计算低精度（Precision）？存储和算术可能有所不同。

8. 还保留哪些元数据？刻度、零点、密码本、异常、填充？

9. 哪个瓶颈得到改善？容量、带宽、计算、延迟（Latency）、吞吐量（Throughput）？

10. 哪个内核（Kernel）/硬件消耗它？如果没有后端支持，压缩可能不会变得速度。

一旦这些问题明确，GPTQ、AWQ、SmoothQuant、LLM.int8()、NF4、FP8 和 KV 缓存方法就更容易比较。它们并不是对同一问题的相互竞争的答案；而是相互竞争的。它们在设计空间中占据不同的位置。

理解框架（Mental Model）。量化（Quantization）是受控的数值近似。有用的问题不是“有多少位？”但是“哪些信息是近似的，用什么地图，什么系统

这种近似能带来什么好处？”

研究依据：[1]-[17]

参考文献

对应原稿第 41 页

主要论文和当前实施来源

1.B.雅各布等人。用于高效整数算术推理（Inference）的神经网络的量化（Quantization）和训练（Training）。 CVPR 2018。

2.M.内格尔等人。神经网络量化（Quantization）白皮书。 arXiv:2106.08295，2021。

3. T. Dettmers、M. Lewis、Y. Belkada、L. Zettlemoyer。 LLM.int8()：大规模 Transformer 的 8 位矩阵乘法（Matrix Multiplication）。 NeurIPS 2022.arXiv:2208.07339.

4.Z.姚等人。 ZeroQuant：针对大型 Transformer 的高效且经济实惠的训练后量化（Post-training Quantization）。 NeurIPS 2022.arXiv:2206.01861.

5.G.肖等人。 SmoothQuant：大型语言模型（Models）的准确高效的训练后量化（Post-training Quantization）。 ICML 2023.arXiv:2211.10438.

6. E. Frantar、S. Ashkboos、T. Hoefler、D. Alistarh。 GPTQ：生成式预训练 Transformer 的准确训练后量化（Post-training Quantization）。 ICLR 2023.arXiv:2210.17323.

7. J.林等人。 AWQ：设备上 LLM 压缩和加速的激活（Activation）感知权重量化（Quantization）。 MLSys 2024.arXiv:2306.00978.

8. T. Dettmers、A. Pagnoni、A. Holtzman、L. Zettlemoyer。 QLoRA：量化 LLMs 的高效微调。 NeurIPS 2023.arXiv:2305.14314.

9. P. Micikevicius 等人。用于深度学习的 FP8 格式。 arXiv:2209.05433，2022。

10.Z.刘等人。 KIVI：键值缓存（KV Cache）的免调整非对称 2 位量化（Quantization）。 ICML 2024.arXiv:2402.02750.

11.C.胡珀等人。 KVQuant：通过 键值缓存（KV Cache）量化（Quantization）实现 1000 万上下文长度 LLM 推理（Inference）。 NeurIPS 2024.arXiv:2401.18079.

12. 邵伟等人。 OmniQuant：大型语言模型（Models）的全方位校准量化（Quantization）。 arXiv:2308.13137，2023。

13. PyTorch 和 torchao 文档。量化（Quantization）观察者、伪量化和当前量化/QAT API。当前文档查阅于 2026 年 9 月。 14. NVIDIA Transformer Engine 文档。 FP8、MXFP8、NVFP4、缩放（Scaling）配方、混合精度执行。当前文档查阅于 2026 年 9 月。 15. NVIDIA TensorRT-LLM 文档。 FP8/FP4、GPTQ/AWQ 和 键值缓存（KV Cache）的量化配方。当前文档参考了 2026 年 9 月。 16.bitsandbytes 文档和源。 NF4/FP4 码本、4 位封装、QLoRA 集成。 当前来源咨询于 2026 年 9 月。 17. D. Yoshida。 NF4 并不是理论上信息最优的（这很好）。 arXiv:2306.06965，2023。